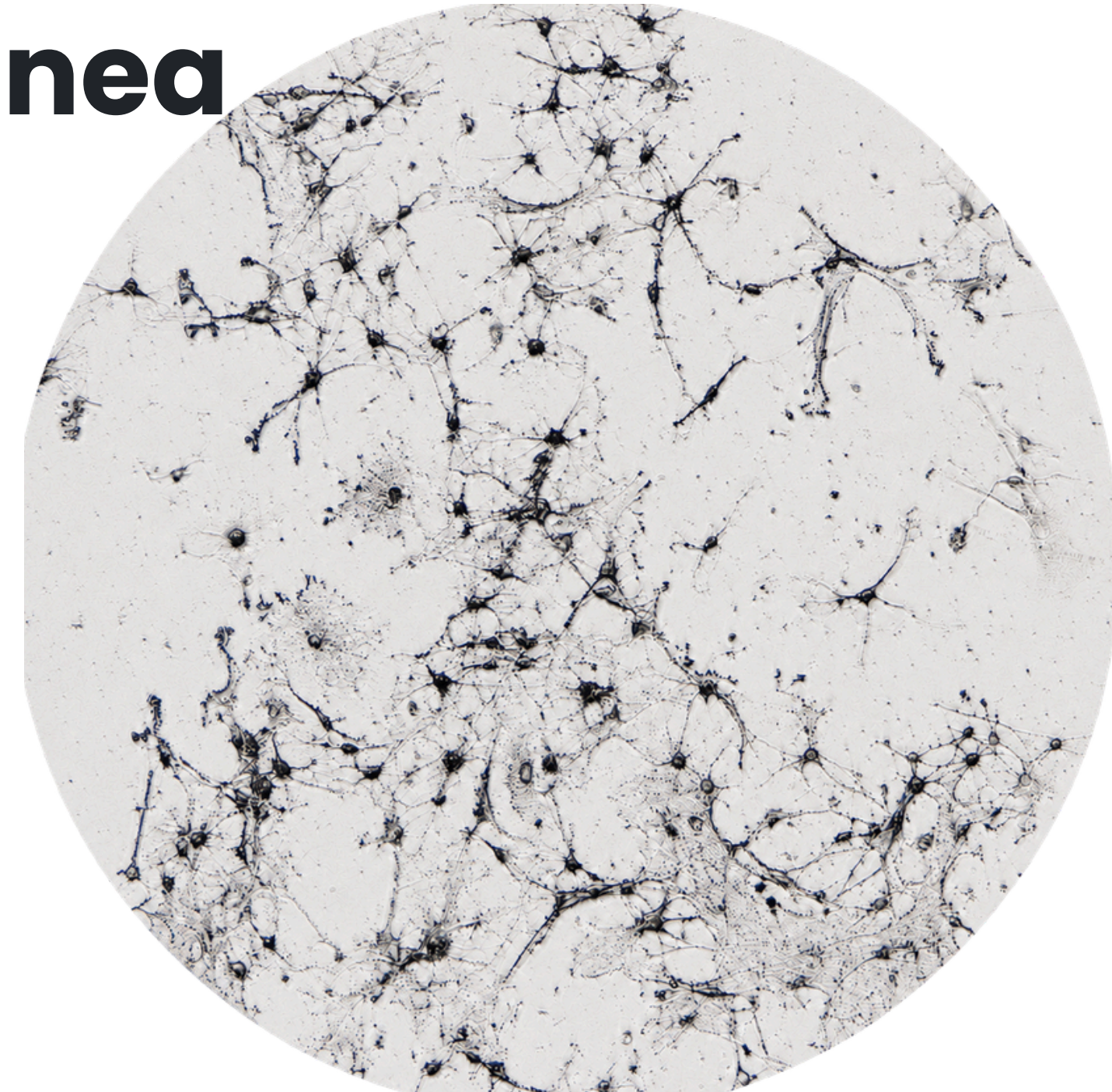


Reimplementação heterogênea (GPU + CPU multicore) do pipeline mbkmeans **para scRNA-seq**

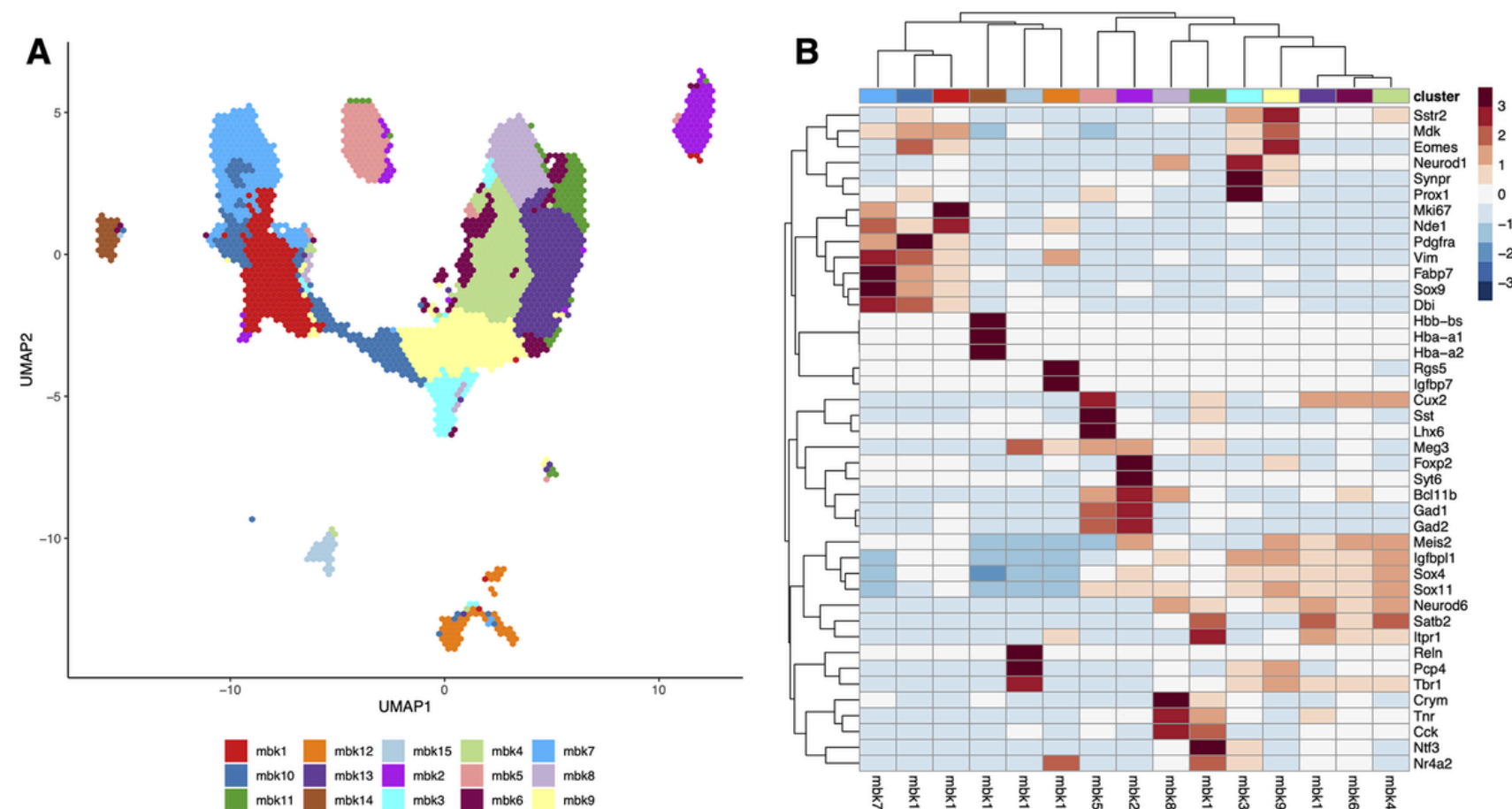
Onde a GPU compensa e o que o speedup revela sobre os dados

Christian de Souza Ramos • Giulio Henrique Borges Basso
Igor Garbin Manzan Mazo • Wallisson Policarpo Teodoro



O problema: clusterizar milhões de células

- ScRNA-seq agrupa células pela expressão gênica para identificar subpopulações celulares, um passo central é o clustering não supervisionado.
- Dado de entrada: matriz esparsa de ~1,3 milhão de células × ~28 mil genes, ~55 GB em HDF5 (formato CSR), com ~2,4 bilhões de não-zeros.
- Nessa escala, o K-means clássico (que carrega tudo na memória) torna-se custoso ou inviável.
- Hicks et al. (2021) propõem o mbkmeans (mini-batch K-means on-disk). O pipeline completo roda em CPU single-thread em ~104 h, dominado pelo PCA-IRLBA (96 h) e pela normalização (5,18 h).



VOLUME DE DADOS

55 GB

matriz esparsa em HDF5

ESCALA

1,3 M

células × 28 mil genes

BASELINE (CPU)

~104 h

single-thread, paper

Cinco etapas encadeadas



Objetivo: reimplementar o pipeline em arquitetura heterogênea, GPU (CuPy/CUDA) para o cálculo pesado e CPU multicore (Numba/OpenMP) para a movimentação de dados.

A GPU é sempre mais rápida?



A GPU só compensa quando o gargalo é o cálculo.

O dado precisa viajar até a GPU: SSD → RAM → VRAM → cálculo.

Se o tempo é gasto movendo dado (I/O de disco ou transferência PCIe), acelerar o cálculo quase não muda o tempo total, a GPU fica ociosa esperando o dado (data starvation).

Ela só rende quando o cálculo é pesado, regular e reutiliza dados já transferidos.

Nossa contribuição: medir, etapa a etapa, onde a GPU compensa e onde a CPU é a escolha certa, análise que o paper (CPU-only) não faz.

A ANALOGIA

"Uma Ferrari (a GPU) presa no engarrafamento (I/O ou PCIe) anda na velocidade do trânsito. Trocar o motor não adianta, o que limita é a estrada."

Híbrido: GPU no cálculo, CPU na movimentação

Pass 1 → GPU

Métricas de QC (contagem por célula, % mitocondrial) via prefix-sums (cumsum) direto sobre os arrays CSR — sem nunca densificar a matriz.

Acesso sequencial e regular: o padrão em que a GPU é eficiente. É aqui que está o ganho legítimo.

Pass 2 → CPU multicore

Montar a matriz filtrada é mover memória, não calcular (Numba + OpenMP, paralelizado por célula).

Escrita irregular (cada célula tem nº diferente de não-zeros) e embaraçosamente paralela, sem race condition: cenário ideal para CPU.

SPEEDUP DA ETAPA

~1,71×

I/O-bound · 8 min → 4 min 40 s

Ganho modesto porque o tempo é dominado por ler ~2,4 bilhões de não-zeros do SSD, o disco não fica mais rápido só porque quem processa é a GPU.

Roda em GPU e mede-se que ela NÃO compensa

Implementado em GPU (CuPy): assignment, somas por cluster e update dos centróides na VRAM. Ainda assim, o ganho é de apenas $\sim 1,17\times$. Isso não é defeito, é um resultado medido.

Iterações repetidas

Cada uma das ~ 2.000 iterações reenvia um novo batch pela PCIe. O QC transfere $1\times$; o clustering, milhares.

Cálculo barato demais

A GPU calcula as distâncias em microssegundos, mas a transferência custa milissegundos $\rightarrow$ data starvation.

Esparsidade (CSR)

A indireção (olhar o índice antes do valor) quebra o acesso coalescido e desperdiça a largura de banda da GPU.

Mostrar onde a GPU NÃO ajuda é parte da contribuição. O gargalo aqui é a transferência PCIe repetida de dados esparsos com acesso irregular, não o cálculo.

SPEEDUP DA ETAPA

$\sim 1,17\times$

transfer-bound + esperso

Reimplementação fiel do scrn na GPU

O paper não descreve a normalização, terceiriza ao scrn (Lun et al., 2016). Por que a GPU compensa ($\neq$ clustering): o cálculo por bloco é pesado e regular (divisões, medianas em janelas, solve) sobre um bloco denso e limitado ($\leq \sim 170$ MB) que cabe na VRAM, sem as idas-e-vindas da PCIe. Por que scrn e não library-size: a library-size assume mesma composição de RNA e apaga sinal biológico real; o scrn compara células parecidas (pooling), por isso depende do clustering inicial.

SPEEDUP DA ETAPA

38,85×

compute-bound · 5,18 h → 8 min

VALIDAÇÃO

Conferido em NumPy contra fatores de tamanho conhecidos:

corr 0,99 (1 cluster) · 0,997 (multi-cluster)

Erro $\sim 2,6\text{--}6\%$, contra $\sim 15\text{--}23\%$ da normalização simples por library size.

Nossa implementação

Mesmo algoritmo de deconvolution, fiel passo a passo: pré-escala, pools circulares (21–101), mediana de razões, solve linear (LSQR) e reescala entre clusters. Mas as somas viram prefix-sums (cumsum) vetorizáveis e o solve roda por bloco denso na VRAM, mesma formulação, caminho casado ao hardware.

Troca de algoritmo: IRLBA → covariância

IRLBA (paper)

SVD truncado e iterativo que relê a matriz do disco a cada iteração. Genérico, mas o preço é I/O: dezenas de passadas pelo disco = 96 h. O gargalo é leitura repetida, não cálculo.

Nosso método (covariância)

Como n (1,1 M) $\gg$ p (13,9 k), a covariância gene×gene (~1,5 GB) cabe na VRAM. Em 2 passadas: $S = X^T X + \text{média} \rightarrow \text{eigh}(M)$ dá os top-50 loadings $\rightarrow$ projeta os scores. Exato: corr 1,000000 vs scikit-learn.

INSIGHT CENTRAL

O maior ganho é ALGORÍTMICO, não da GPU. Reconhecer que $n \gg p$ e trocar o SVD iterativo por covariância em 2 passadas esmagaria o IRLBA mesmo em CPU. A GPU atua como multiplicador (acelera o `eigh` e os `matmuls`).

SPEEDUP DA ETAPA

1080×

96 h $\rightarrow$ 5 min 20 s

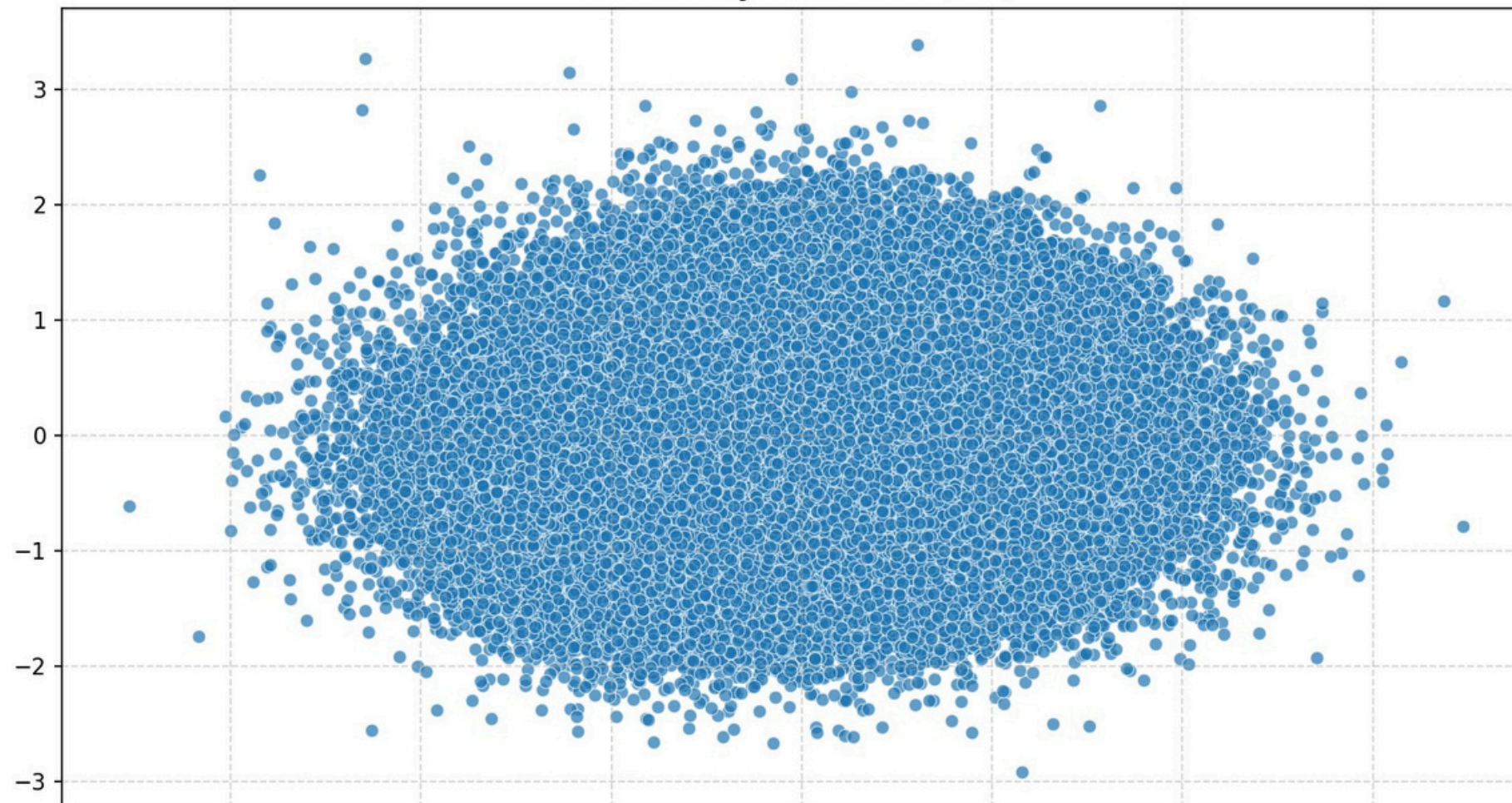
Lloyd completo, pós-PCA, com busca de K

Após o PCA, a matriz é $1,1 \text{ M} \times 50$ (densa, $\sim 220 \text{ MB}$) e cabe inteira na VRAM. Sem motivo de memória para aproximar $\rightarrow$ usamos Lloyd completo, em que cada iteração é um único GEMM $(n \times d) \cdot (d \times k)$: o padrão em que a GPU é imbatível.

O próprio paper mostra que mini-batch $\approx$ K-means exato (batch ≥ 500), então a troca remove a aproximação sem alterar as conclusões biológicas.

Além do paper: o paper fixa $K=15$. Fazemos busca sistemática sobre K , com $n_{\text{init}}=3$ reinícios, selecionando por WCSS/ n (cotovelo) + silhouette, operacionalizando uma sugestão que o paper faz mas nunca executa.

Visualização do PCA (2D)



INICIAL vs FINAL

Clustering inicial

$1,1 \text{ M} \times 13,9 \text{ k}$, esparsa, não cabe na VRAM. Mini-batch obrigatório. GPU NÃO compensa.

Clusterização final

$1,1 \text{ M} \times 50$, densa (220 MB), cabe na VRAM. Lloyd completo. GPU decisiva.

RESULTADOS · DESEMPENHO

~102 h → ~26 min: speedup global ~232×

Etapa		Baseline	Proposta	Speedup
Pré-processamento (QC)		8 min	4 min 40 s	~1,71×
Clustering inicial		8 min 30 s	7 min 14 s	~1,17×
Normalização (scrnan)		5,18 h	8 min	38,85×
PCA		96 h	5 min 20 s	1080×
Clusterização final (busca de K)		~30 s	~70 s	~0,43×
Total (5 etapas)		~102,3 h	~26,47 min	~231,91×

Ambiente: NVIDIA RTX 5070 Laptop (Blackwell, 8 GB VRAM), 24 CPU-cores, 32 GB RAM, SSD NVMe. O baseline são os tempos reportados por Hicks et al. (CPU single-thread); a comparação é entre pipelines, não entre hardwares equivalentes.

O número agregado não é o ponto, o porquê é

Etapa	Gargalo	Natureza	GPU compensa?
Pré-proc. (QC)	SSD → RAM (I/O)	I/O-bound	Pouco
Clustering inicial	PCIe (repetido)	Transfer-bound + esparso	Não
Normalização	Cálculo por bloco	Compute-bound	Sim
PCA	Algorítmico	Compute-bound, denso	Sim, muito
Clusterização final	Cálculo denso	Compute-bound, denso	Sim, decisivo

A mesma GPU dá ~1,7× num caso e ordens de magnitude no outro. Distinguir I/O-bound × transfer-bound × compute-bound, e alocar GPU ou CPU de acordo, é o cerne do trabalho.

A aceleração não comprometeu a corretude

NORMALIZAÇÃO (scrn)

corr 0,99 / 0,997

1 cluster e multi-cluster vs. fatores de tamanho conhecidos. Erro ~2,6–6% (contra ~15–23% da library-size simples).

PCA

corr 1,000000

Validado contra o PCA do scikit-learn nos 50 componentes, reformulação matematicamente exata.

INTEGRIDADE PONTA A PONTA

1.148.558 células

Validador em streaming (bruto → filtrado → clusterizado → normalizado → PCA): 0 barcodes duplicados, 13.897 genes consistentes.

A consistência ponta a ponta reforça que o silhouette baixo (próximo slide) é estrutura real do dado, não desalinhamento entre etapas do pipeline.

WCSS vs. Silhouette — métrica ausente no paper

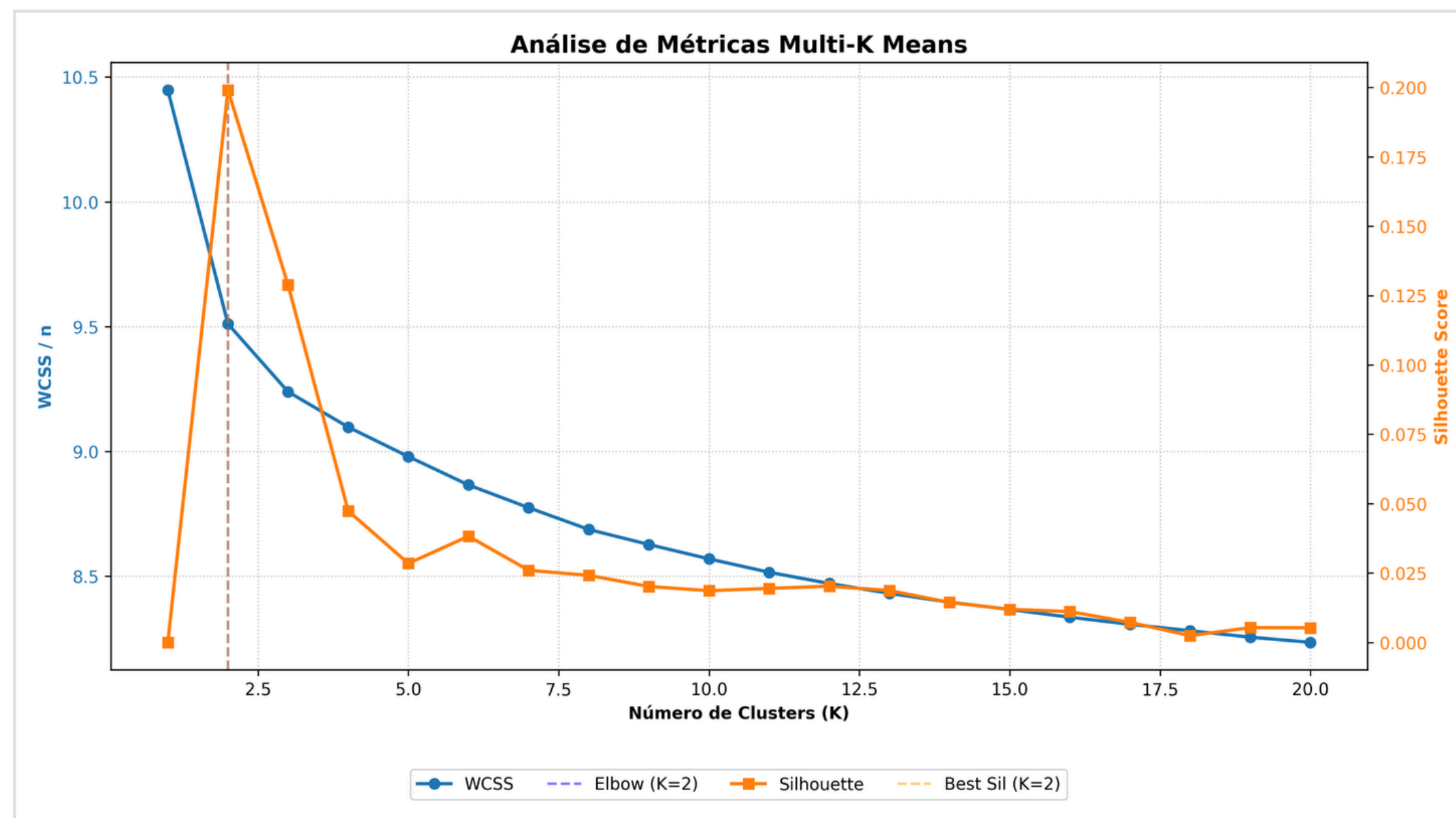
O paper avalia o agrupamento só por WCSS, que decresce monotonicamente com K e não mede a separação dos clusters.

Usamos WCSS/n que contém o mesmo comportamento monotônico que o WCSS, porém é o indicado para quando se tem toda a integralidade dos dados.

Adicionamos o silhouette: pico isolado em K=2 (~0,20) e colapso para ≈ 0 nos demais valores de K.

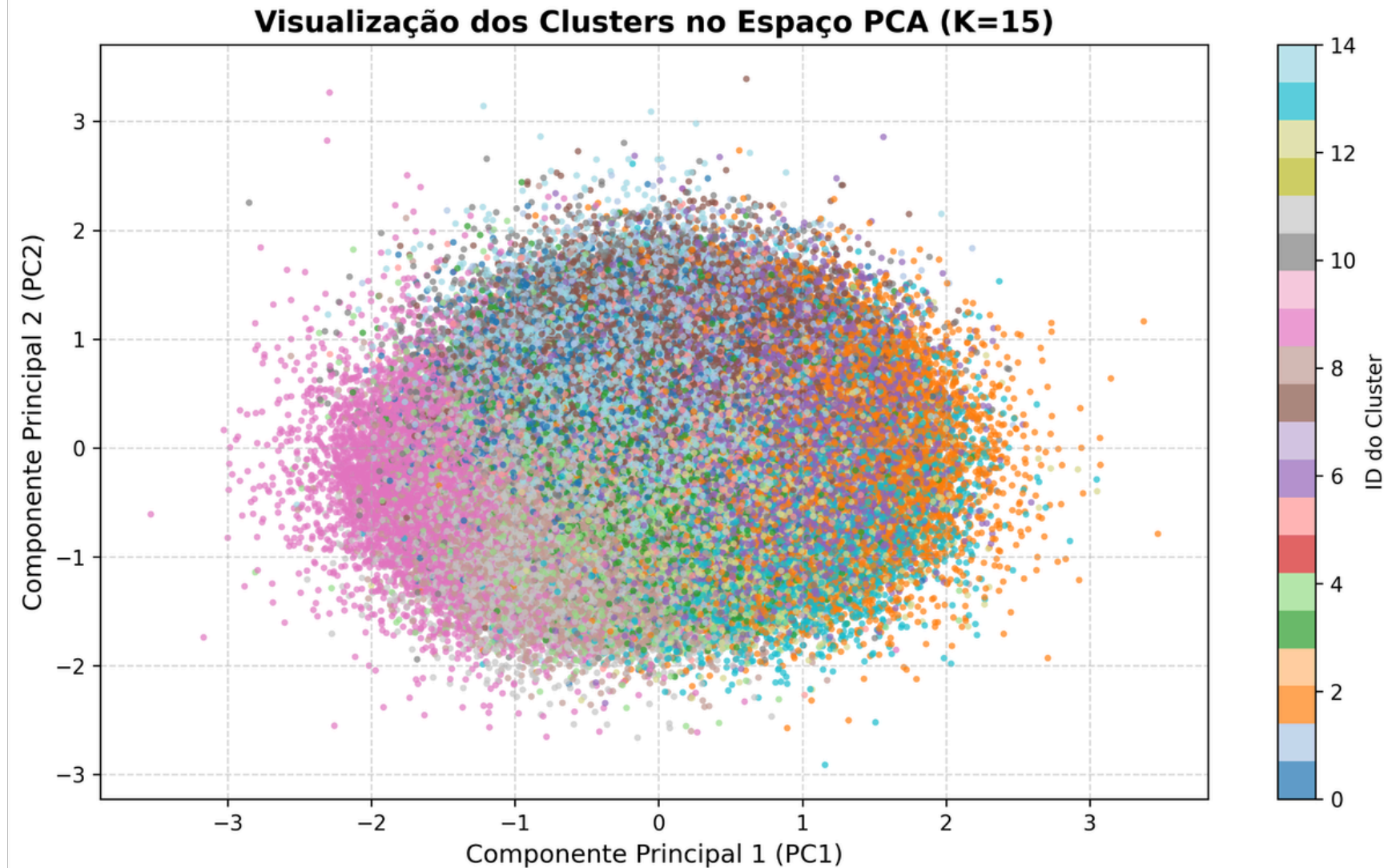
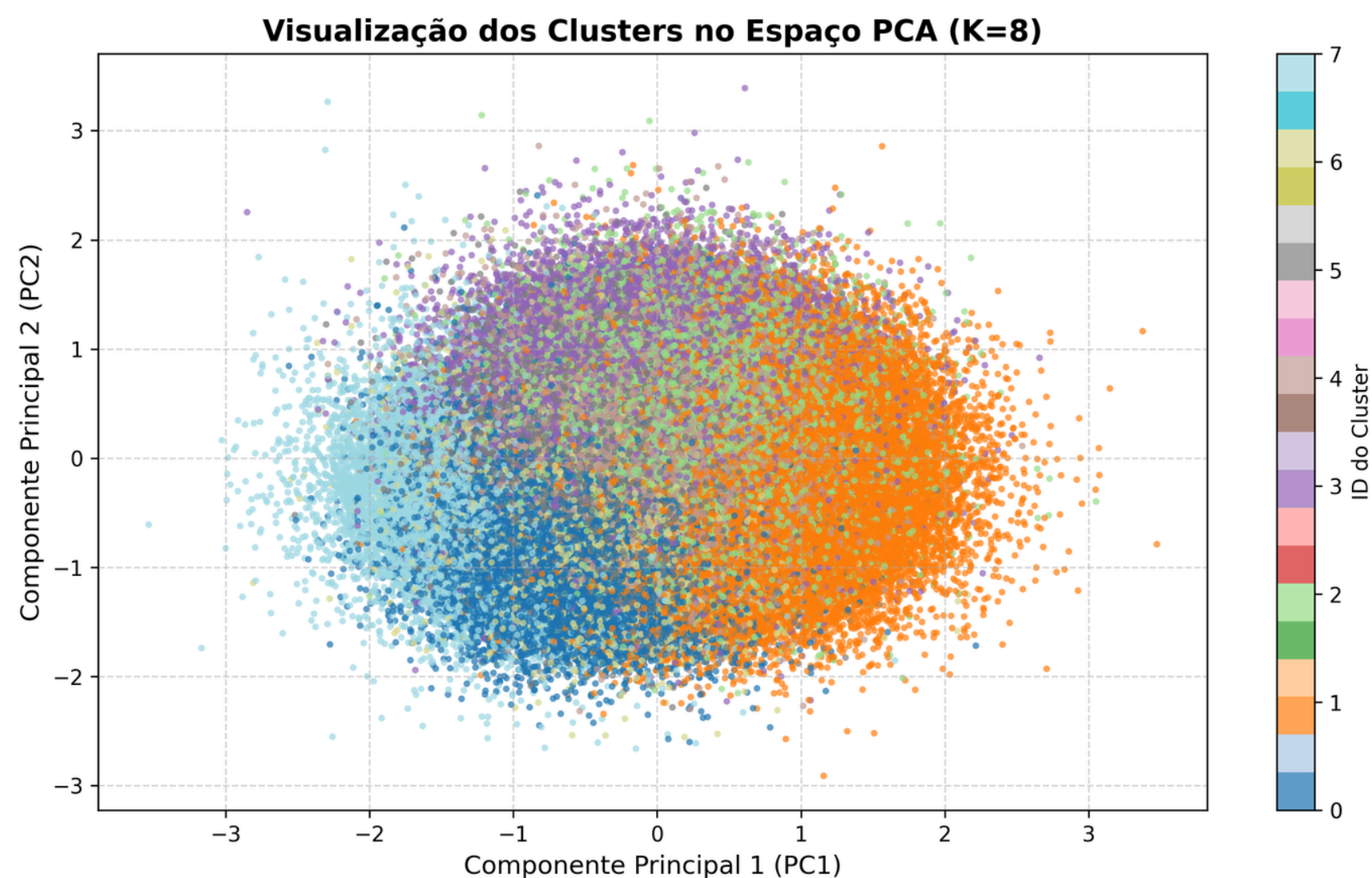
O padrão se manteve entre rodadas independentes.

Convenção: silhouette < 0,25 indica estrutura de cluster fraca ou ausente.



Análise multi-K: WCSS/n e silhouette para K de 1 a 20.

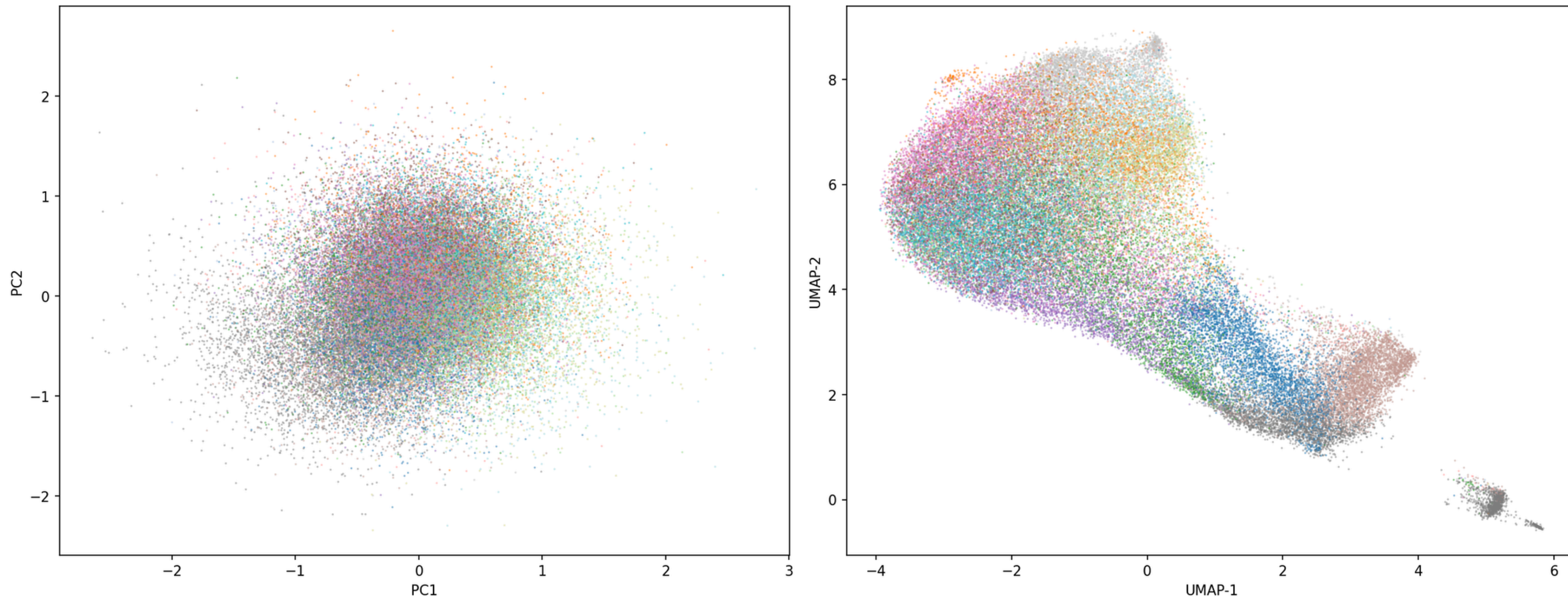
Aumentar K não cria fronteiras



Projeção dos clusters no espaço PCA (PC1 × PC2): K=8 (esq.) e K=15 (dir.), uma nuvem contínua, sem fronteiras nítidas.

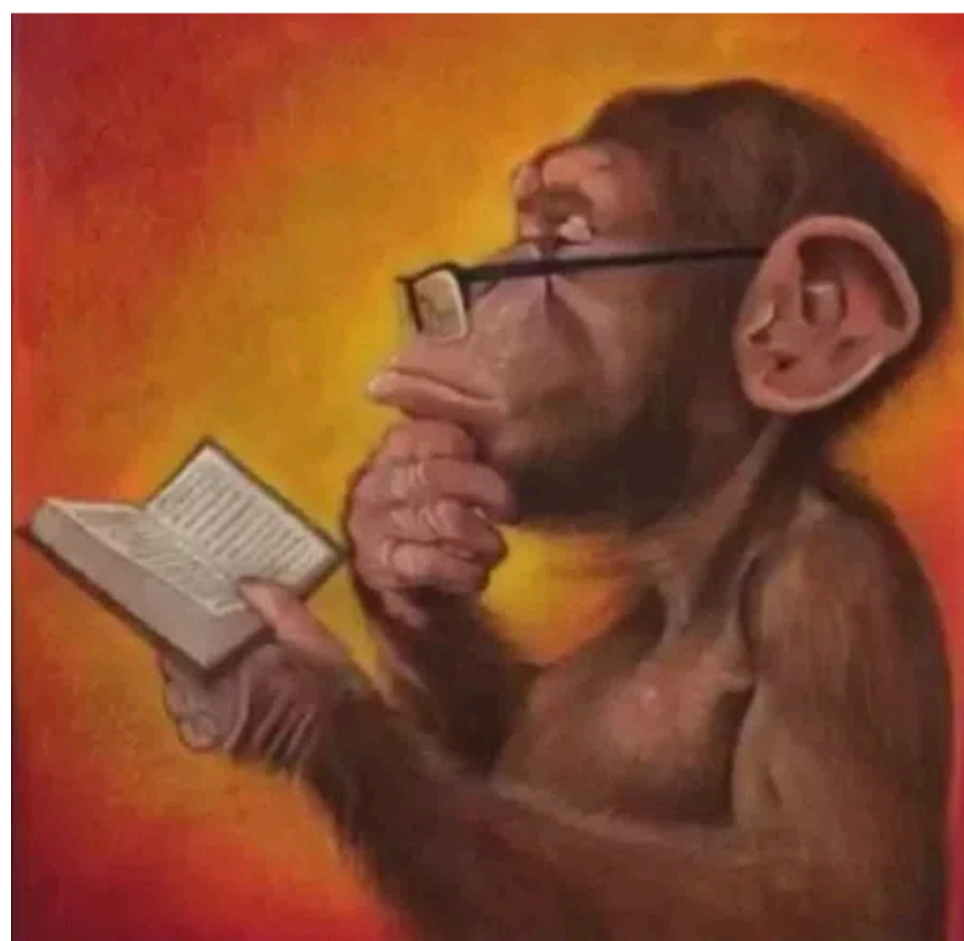
Hipótese: neurônios formam um contínuo de diferenciação, não clusters discretos. O K-means (distância euclidiana) assume clusters globulares, num gradiente, qualquer K dá silhouette baixo. Não invalida a implementação; é limitação do método frente ao dado.

Dados distribuídos de forma contínua (UMAP)



Projeção dos clusters no espaço PCA (PC1 x PC2): K=15 (esq.) e UMAP(UMAP-1 x UMAP-2): K=15 (dir). Uma nuvem contínua, sem fronteiras nítidas.

Levando em consideração a estrutura não esférica dos dados biológicos é plausível pensar que os dados de neurônios formam um contínuo de diferenciação que poderia ser melhor capturado por sobreposições de elipsoides. Desta forma uma possível solução para o problema seria clusterizar usando misturas de gaussianas(GMM) ou métodos baseados em grafos.



*O que esperávamos
neste trabalho?*



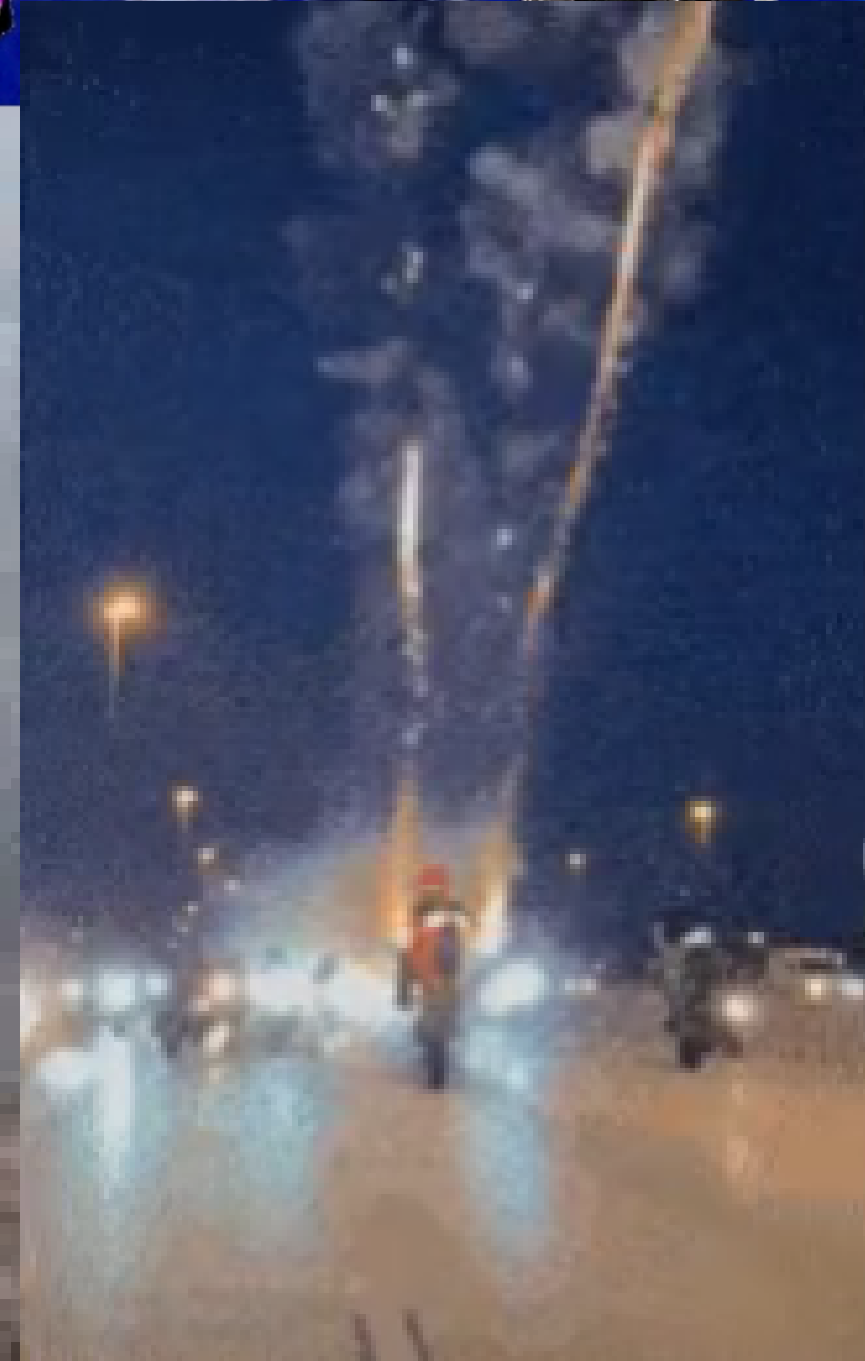
SPEEDUP 20X





O que conseguimos?

SPEEDUP 232X



CONCLUSÃO

O que levamos deste trabalho



232×

Speedup global reimplementando as 5 etapas em arquitetura heterogênea (GPU + CPU multicore): ~102 h → ~26 min.

Onde

A GPU só compensa quando o gargalo é cálculo pesado e regular que cabe na VRAM, documentamos até um caso onde ela não compensa (clustering inicial).

Algoritmo

O maior ganho (PCA, 1080×) é majoritariamente algorítmico; a GPU atua como multiplicador, não como fonte primária do ganho.

Silhouette

A métrica ausente no paper revela que os dados não são bem separados por K-means, estrutura mais contínua que discreta.